A Small Nervous System as Sequencer: Neuroevolved Spiking Networks for Three-Voice Pentatonic Music

Anonymous Author(s)
Submission to the AI Music Composition Competition, AIMG 2026

***Abstract*—We describe a compositional system in which a small spiking neural network is run as an autonomous dynamical system and shaped by evolutionary search until its spike activity, read out as notes, forms three-voice music in a Japanese pentatonic scale. The network receives no input beyond an initial stimulus and is not trained on musical data. Each of three readout layers selects one of five scale degrees per time step, so that every note the network can produce lies in the scale by construction. A $(\mu + \lambda)$ evolution strategy with Gaussian and differential mutation optimizes connection weights, firing thresholds, and refractory periods against a multiplicative fitness function that rewards a target note density, pitch variety, and the absence of repeated figures, and, in a later version, agreement with a target distribution of melodic transitions and the placement of stable tones on strong beats. The submitted work is a piece in two movements. The first was produced by a network of three loosely coupled modules, each driving one voice, in the Kumoi scale; the second by a network with a single shared pool of neurons in the Iwato scale. The evolved MIDI files were not edited. Orchestration, including the doubling of voices across instruments, and all real-time MIDI and audio effects, including a slow drift in note timing on two voices, audio echo, and, in one movement, a reversed copy of one voice, as well as tempo and mixing, were decided by a human in a digital audio workstation. We describe the system, the failure modes that shaped it, and the division of labor between algorithm and interpreter.**

***Index Terms*—spiking neural networks, neuroevolution, reservoir computing, algorithmic composition, symbolic music evaluation, pentatonic scales, human–AI co-creation**

I. Introduction

Most generative music systems based on neural networks are trained to approximate a mapping from an input to an output, and composition is performed by sampling from the trained model. This work takes a different position. The network is treated as a physical system with its own dynamics, and the question is whether evolutionary pressure can shape those dynamics into something a listener would accept as music. The composition is encoded in the weights and topology of the network rather than in any input or dataset.

The architecture belongs to the family of recurrent networks studied in reservoir computing. Maass, Natschläger, and Markram [1] proposed that a recurrent circuit of spiking neurons, the "liquid," can serve as a generic high-dimensional dynamical system from which readout units extract useful signals. Jaeger's echo state networks [2] apply the same separation to rate-based units, and Jaeger also demonstrated that a trained echo state network can generate signals autonomously, without input. In both frameworks the reservoir is random and fixed, and the readout is trained by regression against target data. In the present system no musical training data is used: evolution determines both the reservoir and the readout, and the network runs freely after a single initial stimulus. Kiefer [3] examined echo state networks as mapping tools for musical instruments and treated the spectral radius of the reservoir as a controllable gain; our observations about the role of inhibitory connections and refractory periods are in the same spirit. The Continuous Thought Machine of Darlow et al. [4] motivated our early experiments with neuron synchronization as a readout signal.

The search procedure is a conventional evolution strategy [5] augmented with differential mutation [6]. As in several earlier neuroevolution methods, offspring are produced by mutation only [7, §2.1.3].

The division of labor between algorithm and interpreter was inspired by John Cage, some of whose chance compositions leave decisions to the performer [8]. Here the network's MIDI output is the score, and a human interprets it.

The contributions of this work are modest and practical. First, a spiking reservoir is used as an autonomous sequencer, with evolution replacing readout training. Second, the pitch scale is built into the readout layer rather than imposed as an objective, which removes a conflict between tonal consistency and pitch diversity that otherwise stalls the search. Third, a compact fitness function compares each voice's full distribution of melodic transitions against a hand-specified target for the scale, which resists exploits that defeated simpler metrics. Fourth, a modular topology with a single coupling parameter yields voices with distinct rhythmic behavior while the network remains one organism under one fitness function.

II. System

A. Network

The network consists of simple threshold neurons connected by a sparse random weight matrix with both excitatory and inhibitory weights. At each discrete time step, a neuron accumulates weighted input from the neurons that fired on the previous step, its accumulated activation decays by a fixed leak factor, and it fires when activation reaches its threshold. After firing, the neuron enters a refractory period during which it accepts no input and its activation decays more rapidly. Refractory periods are assigned per neuron from a

uniform distribution and are themselves subject to mutation. The complete update rule is given in Appendix A, and all parameter values are listed in Appendix B.

Two parameters dominate the network's temporal behavior. The activation leak determines whether activity can persist across silences; a leak factor of 0.98 per step allows activity to survive gaps of tens of steps, whereas a factor of 0.90 does not. The distribution of refractory periods determines the rhythmic texture; a wide spread of periods across neurons produces activity on many time scales. Inhibitory connections proved more effective than tuning of the spectral radius for producing varied, non-saturating dynamics.

The simulation runs for 288 steps, interpreted as sixteenth notes (72 beats), after a single initial stimulus applied to the most strongly connected neuron. No further input is provided.

B. Readout and encoding

Three readout layers are attached to the network, one per voice. Each readout has five output units, one per degree of the chosen pentatonic scale, connected to the reservoir through a sparse random weight pattern that is also evolved. Output units integrate the firing of their connected neurons with a fixed decay. At each step, the output unit with the highest value in a readout sounds its scale degree, provided its value exceeds a fixed threshold of 0.3; the note's MIDI velocity is proportional to that value. If the same degree wins on consecutive steps, the note is sustained rather than restruck. Each voice is confined to its own octave.

Two properties of this encoding mattered. Because the output units are scale degrees, every note is in the scale by construction, and tonal consistency need not appear in the fitness function. Because the trigger threshold is fixed rather than adaptive, the network controls its own note density. Earlier versions used percentile-based or peak-relative thresholds, which guaranteed a certain density regardless of network behavior and prevented evolution from discovering sparse textures.

The system supports the In-Sen (C, D♭, F, G, B♭), Iwato (C, D♭, F, G♭, B♭), and Kumoi (C, D, E♭, G, A) scales; changing the scale requires changing one line of the configuration.

C. Unified and modular configurations

In the unified configuration, all three readouts draw from a single pool of 120 neurons. The resulting voices are coherent but tend to move together. In the modular configuration, the network has 255 neurons partitioned into three contiguous blocks of 85, each voice's readout is restricted to one block, and a single parameter sets the density of connections between blocks relative to the density within them; the submitted movement used a ratio of 0.5. The network is still evolved as a single genome under a single fitness value, and the connections between blocks are shaped by selection like any others. As a diagnostic, we track the mean pairwise absolute correlation of the blocks' firing rates; it is determined largely by the topology and is not part of the fitness.

III. Evolution and Fitness

A. Search

The search is a $(\mu + \lambda)$ evolution strategy with $\mu = 24$ parents and $\lambda = 120$ offspring per generation, run for 88 generations. The genome comprises the reservoir weights, the readout weights, the firing thresholds, and the integer refractory periods. Three fifths of the offspring are produced by Gaussian mutation of a single parent, divided among five groups that anneal the mutation rate or scale upward or downward as a function of how many generations the parent has survived without improvement. The remaining two fifths are produced by differential mutation: the base parent is moved by a scaled difference between two other parents of unequal fitness. The differential step is large while the population is diverse and shrinks as it converges. No crossover and no random immigrants are used. Search settings were identical for both configurations.

B. Fitness

Fitness is computed from the MIDI output as a product of terms, each in $[0, 1]$. Every term except activity is computed per voice, and the minimum across voices is taken, so that a single deficient voice determines the score. The product form means each term acts as a gate: a voice that excels on one term and fails another scores near zero. Definitions are given in Appendix C.

Two versions of the fitness function were used. The unified configuration was evolved under

$$F_{\text{unified}} = \text{activity} \times \text{diversity}. \tag{1}$$

Activity measures the total note count against a target equal to the number of simulation steps, decreasing linearly with the absolute log-ratio between the two, so that silence and saturation are penalized symmetrically. *Diversity* is the mean of two components: the normalized entropy of the voice's pitch-class distribution, and a *repetition* score that penalizes a voice whose most frequent two- or three-note sequence accounts for a large share of all its sequences, with two occurrences permitted before the penalty applies.

The modular configuration was evolved under

$$F_{\text{modular}} = \text{activity} \times \text{tonal} \times \text{repetition} \times \text{metric}. \tag{2}$$

Pitch entropy was replaced by *tonal gravity*, which compares the distribution of a voice's melodic transitions, over the twenty ordered pairs of distinct scale degrees, with a target distribution written by hand for each scale. The targets encode the characteristic motion of the scale; for Kumoi, the half-step between D and E♭ is weighted against the structural fifth C–G. Agreement is measured as the exponential of the negative Kullback–Leibler divergence from target to observed, with additive smoothing. Because the target assigns probability to all twenty transition types, a voice that uses only a few is heavily penalized, so the term also enforces pitch variety. *Metric gravity* scores a voice's tendency to place stable tones (C and G for Kumoi) on metrically strong sixteenth-note

positions, weighted by a binary-subdivision pattern over a four-beat cycle. The score peaks when about half of the strong-position notes are stable tones, so that neither ignoring the meter nor playing the root on every downbeat is rewarded. Metric gravity was added because the more independent voices of the modular network lacked a shared sense of downbeat and tonal center.

C. Design history

The fitness function and the encoding reached these forms through a series of failures, which we report because they are as informative as the final design.

The first fitness function combined eleven metrics from the computational musicology literature with hand-tuned weights. Within 37 generations the best network outscored a set of reference ambient recordings, including Brian Eno's *Music for Airports*, on this function. The function was discarded as over-specified.

With twelve chromatic output units per voice, a tonal-consistency term and a pitch-diversity term were in direct conflict: every additional pitch class in use was another opportunity to leave the scale. Evolution settled on a trade-off that satisfied neither. Reducing the output layer to the five scale degrees removed the conflict and the tonal-consistency term together.

A global diversity measure was satisfied by networks in which each voice repeated a single note, since the voices chose different notes. Computing the measure per voice and taking the minimum closed this exploit.

A first version of tonal gravity scored each transition individually against the target row for its origin. Networks converged on two-note oscillations between C and D♭, scoring 0.94 on the term while producing music no one would want to hear. The term could not detect that eighteen of twenty transition types never occurred. Comparing the full joint distribution corrected this.

Finally, lineage tracking showed that one ancestral line typically dominated the parent pool within five generations. Speciation and fitness sharing were tried and did not change the music. The limiting factor was the fitness landscape, not the diversity of the population.

IV. The Submitted Work

A. Two movements

The submitted piece consists of two movements of four minutes each. The first, *The Enduring Enso*, was produced by the modular configuration in the Kumoi scale. The second, *Echo Chamber*, was produced by the unified configuration in the Iwato scale. The unified configuration was developed first; presenting the modular movement first is an artistic decision.

Each configuration was run as a batch with a fixed random seed per run: twelve runs (four per scale) for the unified configuration and nine runs (three per scale) for the modular configuration. Each run yields one composition, its best-scoring MIDI file in the final generation. No intermediate or lower-ranked outputs were considered. Selection among runs was made by listening: for each movement, the compositions from several runs in the same scale were auditioned in the digital audio workstation and one was kept. The chosen runs were not the highest-scoring in their scale. The recorded fitness of the first movement is 0.79 and that of the second is 0.91; because the two were evaluated under different fitness functions, the values are not comparable.

The MIDI output is sparse and rhythmically uneven, with bursts of activity separated by silences, which suits a tradition in which silence carries structural weight.

B. Interpretation

The MIDI files are treated as scores and were imported into a digital audio workstation without modification. All further decisions were made by a human interpreter and fall into the following categories.

Tempo. The scores were played at 20 BPM, so that the 72-beat score lasts 3 minutes 36 seconds. The remaining 24 seconds of each movement are a fade-out that separates the movements.

Orchestration. The same orchestration was used in both movements. The first voice is played by a koto. The second voice is doubled by a koto and a shamisen, and the third by two flutes. The shamisen receives its voice through a counter-based MIDI device that drops one note in every twelve in the first movement and one in every nine in the second. This device removes notes from what one instrument plays; it does not add, move, or alter notes. The shamisen is also more sparse than the koto it doubles for a second reason: the chosen instrument responds strongly to note velocity, which the network sets from the strength of the winning output (Section II-B), so low-velocity notes are faint or nearly inaudible. No velocity remapping or filtering was applied, but the choice of this instrument acts as an implicit mapping from velocity to presence. Together, the note dropping and the velocity response make the shamisen an intermittent shadow of the koto line rather than a strict doubling.

Timing drift. On the first and second voices, in both movements, every note is delayed by a few milliseconds through a MIDI delay device whose delay time is modulated by a slow low-frequency oscillator. The effect is a small, continuously varying offset from the sixteenth-note grid, similar to the timing variation of a human player. The third voice is played on the grid. The order and grid positions of notes in the score are unchanged; only their sounding times are displaced.

Real-time effects. Audio processing includes reverb, delay, saturation, filtering, and echo, with some device parameters modulated by low-frequency oscillators. In the second movement, an audio rendering of the third voice was reversed and mixed in alongside the forward voice, so that the flute line is also heard backward.

Mix. The voices were balanced and placed in the stereo field.

The note data of the score is therefore identical to the network's output, while the sounding result includes doublings, omissions within doublings, small timing offsets on

two voices, audio echoes, and, in the second movement, a reversed copy of one voice, all introduced by the interpreter. The fitness function evaluated symbolic MIDI only and was never applied to audio, and no production decision was iterated against any metric.

We consider this division of labor part of the method rather than a departure from it. The compositions have an internal logic of repetition with variation and of voices converging and diverging, but the choice of weight, color, and space for each voice is what makes them legible as music. The premise of the larger project here is that the notes and the sound can be separated, with computation responsible for the former and a human interpreter for the latter.

C. Listening notes

In the first movement, the three voices maintain distinct rhythmic identities and are anchored by C and G on strong beats; the coupling between modules can be heard as occasional entrainment between voices. In the second movement, the voices share a single pool of neurons and tend to move together, converging and diverging as a group, and the two half-step pairs of Iwato (C–D♭ and F–G♭) recur as points of tension. In both, the silences between bursts are the primary formal device.

V. Discussion

Several conclusions held across the project. The network's physical parameters, namely leak, refractory period distribution, sparsity, and weight scale, determined the reachable musical space more than any property of the search; a change in the leak factor from 0.90 to 0.98 had a greater effect than any mutation scheme. The encoding of network state as notes had a greater effect than the fitness function, and building the scale into the readout was the single most consequential change. Evolution of a chaotic spiking system is difficult because most mutations are destructive; mutation scales had to be set more than an order of magnitude below typical weight values, and differential mutation helped by following the structure of the population rather than sampling noise. Every addition of complexity, including more metrics, more output units, adaptive thresholds, and speciation, degraded the results.

The limitations are plain. The pieces have local coherence but no large-scale form; there is no development or arc, which is acceptable for a static, meditative idiom and would not be elsewhere. The two movements were evolved under different fitness functions and network sizes, so they document two stages of the system rather than a controlled comparison. The transition targets encode the author's understanding of the scales and are a form of curation, not learned style. The fitness terms are proxies, and no listening study was conducted.

AI Usage Disclosure

An AI coding assistant was used throughout the development of the system, for writing and refactoring code, and for discussing design options, evaluation metrics, and related literature; design decisions and evaluation of the results by listening were made by the author. The assistant also helped summarize experimental logs into lab notes. This paper was co-written with an AI assistant from the author's notes, experiment outputs, and prior writing, and the final text was co-edited by the author and the assistant. All parameters, results, and references were verified by the author against the code, the experiment records, and the cited sources. No AI tools were used to generate or process the audio; the music was produced from the network's output with conventional software instruments and effects.

References

[1] W. Maass, T. Natschläger, and H. Markram, "Real-time computing without stable states: A new framework for neural computation based on perturbations," *Neural Computation*, vol. 14, no. 11, pp. 2531–2560, 2002.

[2] H. Jaeger, "The 'echo state' approach to analysing and training recurrent neural networks," GMD Report 148, German National Research Center for Information Technology, 2001.

[3] C. Kiefer, "Musical instrument mapping design with echo state networks," in *Proc. Int. Conf. New Interfaces for Musical Expression (NIME)*, London, 2014, pp. 293–298.

[4] L. Darlow, C. Regan, S. Risi, J. Seely, and L. Jones, "Continuous thought machines," in *Advances in Neural Information Processing Systems*, 2025, arXiv:2505.05522.

[5] H.-G. Beyer and H.-P. Schwefel, "Evolution strategies: A comprehensive introduction," *Natural Computing*, vol. 1, no. 1, pp. 3–52, 2002.

[6] R. Storn and K. Price, "Differential evolution: A simple and efficient heuristic for global optimization over continuous spaces," *Journal of Global Optimization*, vol. 11, no. 4, pp. 341–359, 1997.

[7] K. O. Stanley and R. Miikkulainen, "Evolving neural networks through augmenting topologies," *Evolutionary Computation*, vol. 10, no. 2, pp. 99–127, 2002.

[8] J. Pritchett, *The Music of John Cage*. Cambridge, U.K.: Cambridge University Press, 1993.

APPENDIX A
UPDATE RULE

One simulation step, as implemented. W is the reservoir weight matrix, W_{out} the readout weight matrix, a the activation vector, f the Boolean firing vector, `ok` the mask of non-refractory neurons, r the refractory counter vector, p the per-neuron refractory period vector, and o the output vector.

```
incoming = W.T @ f
ok       = (r == 0)
a        = where(ok, clip(a + incoming, 0, 1), a)
f        = ok & (a >= thresholds)
r_new    = where(f, p, r)
a        = where(r > 0, a * refraction_leak, a)
o        = o * refraction_leak + W_out.T @ f
a        = a * activation_leak
r        = max(0, r_new - 1)
```

Per readout, the step's note is the scale degree at $\arg\max$ of the readout's outputs if the maximum exceeds 0.3, with velocity $\mathrm{round}(127 \cdot \max)$; otherwise the voice is silent. A degree that wins on consecutive steps is sustained.

APPENDIX B
PARAMETERS

Parameter	Mvt. 1 (modular)	Mvt. 2 (unified)
Scale	Kumoi	Iwato
Neurons	255 (3×85)	120
Modules	3	1
Inter-/intra-module density ratio	0.5	—
Readouts × outputs	3×5	3×5
Reservoir sparsity / weight std.	0.2 / 0.3	0.2 / 0.3
Readout sparsity / weight std.	0.05 / 0.3	0.05 / 0.3
Activation leak	0.98	0.98
Refraction leak	0.75	0.75
Refractory period (steps)	$2 + U\{0, \ldots, 61\}$	$2 + U\{0, \ldots, 61\}$
Trigger threshold	0.3	0.3
Initial stimulus	most-connected neuron	most-connected neuron
Steps (sixteenth notes)	288	288
Tempo, evaluation / playback	60 / 20 BPM	60 / 20 BPM
μ / λ / generations	24 / 120 / 88	24 / 120 / 88
Weight mutation rate / scale	0.1 / 0.01	0.1 / 0.01
Threshold mutation rate / scale	0.1 / 0.01	0.1 / 0.01
Refractory mutation rate	0.1	0.1
Annealing factor / max. multiplier	0.1 / 10	0.1 / 10
Gaussian : differential, F	3 : 2, 0.7	3 : 2, 0.7
Fitness	act × ton × rep × met	act × div
Runs in batch	9 (3 per scale)	12 (4 per scale)
Orchestration (voices 1 / 2 / 3)	koto / koto + shamisen / two flutes	same, plus reversed flute audio
Note-skip on shamisen	1 in 12 dropped	1 in 9 dropped
LFO timing drift	voices 1, 2	voices 1, 2
Recorded fitness	0.79	0.91

APPENDIX C
FITNESS DEFINITIONS

Every term except activity is computed per voice and aggregated by the minimum across voices.

Activity. With n the total note count across voices and $T = 288$ the target,

$$\text{activity} = \max\bigl(0,\ 1 - \tfrac{1}{2}\,|\log_2(n/T)|\bigr)\,, \tag{3}$$

so that a count of $T/2$ or $2T$ scores 0.5 and $T/4$ or $4T$ scores 0.

Repetition. For $k \in \{2, 3\}$, let $c_{\max}$ be the count of the most frequent k-gram of pitch classes in the voice and N the number of k-grams. With two occurrences allowed,

$$\pi_k = \left(\frac{\max(0,\ c_{\max} - 2)}{N - 2}\right)^2, \tag{4}$$

$$\text{repetition} = \tfrac{1}{2}\left[(1 - \pi_2) + (1 - \pi_3)\right]. \tag{5}$$

Diversity (movement 2). With H the Shannon entropy in bits of the voice's pitch-class distribution,

$$\text{diversity} = \tfrac{1}{2}\min\left(1, \frac{H}{\log_2 7}\right) + \tfrac{1}{2}\,\text{repetition}. \tag{6}$$

The normalization by $\log_2 7$ is carried over from the earlier seven-note version of the system; for a five-note scale the entropy component cannot exceed $\log_2 5 / \log_2 7 \approx 0.83$.

Tonal gravity (movement 1). Let q be the target distribution over the 20 ordered pairs of distinct scale degrees, and p the observed distribution of a voice's transitions between distinct degrees, with additive smoothing $\alpha = 1$. A voice with fewer than four notes or fewer than eight such transitions scores 0. Otherwise

$$\text{tonal} = \exp(-D_{\mathrm{KL}}(q \,\|\, p)) = \exp\left(-\sum_i q_i \log \frac{q_i}{p_i}\right). \tag{7}$$

The Kumoi target is given in Table I.

TABLE I
KUMOI TRANSITION TARGET (ROWS: FROM; COLUMNS: TO)

	C	D	E♭	G	A
C	—	0.30	0.15	0.35	0.20
D	0.25	—	0.40	0.20	0.15
E♭	0.20	0.45	—	0.20	0.15
G	0.30	0.15	0.10	—	0.45
A	0.25	0.15	0.15	0.45	—

Metric gravity (movement 1). Sixteenth-note positions within a four-beat cycle receive weights

$$w = (4, 0, 1, 0, 2, 0, 1, 0, 3, 0, 1, 0, 2, 0, 1, 0).$$

Let ρ be the weighted fraction of a voice's notes on positions with $w > 0$ whose pitch class is a stable tone (C or G for Kumoi). Then

$$\text{metric} = \begin{cases} \rho / 0.5 & \rho \le 0.5, \\ 1 - (\rho - 0.5) & \rho > 0.5. \end{cases} \tag{8}$$

Both recorded fitness values were reproduced by re-evaluating the submitted MIDI files with the evaluator code in effect at the time of each run.